Commonly Used Functions in DIPLib

IAAIM Team

Contents

1 Loading Images **3**
1.1 ImageReadTIFF . . . 3

2 Histograms and Transformations **3**
2.1 Histogram . . . 3
2.2 ContrastStretch . . . 3
2.3 HistogramEqualization . . . 4
2.4 HistogramMatching . . . 4
2.5 ColorSpaceManager . . . 4

3 Segmentations **4**
3.1 Threshold . . . 4
3.2 IsodataThreshold . . . 5
3.3 RangeThreshold . . . 5
3.4 FixedThreshold . . . 5
3.5 VolumeThreshold . . . 6
3.6 HysteresisThreshold . . . 6
3.7 MorphologicalThreshold . . . 6
3.8 Entropy . . . 6
3.9 OtsuThreshold . . . 7
3.10 KMeansClustering . . . 7
3.11 Label . . . 7
3.12 GradientMagnitude . . . 8
3.13 SobelGradient . . . 8
3.14 Laplace . . . 8
3.15 Canny . . . 9

4 Mathematical Morphological Functions **9**
4.1 Dilation . . . 9
4.2 Erosion . . . 9
4.3 Opening . . . 10
4.4 Closing . . . 10
4.5 AreaOpening . . . 10
4.6 AreaClosing . . . 10
4.7 BinaryOpening . . . 11
4.8 BinaryClosing . . . 11
4.9 GrowRegions . . . 11
4.10 SplitRegions . . . 12
4.11 MorphologicalRange . . . 12
4.12 Tophat . . . 12
4.13 HitAndMiss . . . 13
4.14 Metric . . . 13
4.15 VectorDistanceTransform . . . 13
4.16 ConvexHullInterval2D . . . 14

4.17 Thickening . . . 14
4.18 MeasurementTool . . . 14
4.19 Measure . . . 14
4.20 EdgeObjectsRemove . . . 15

5 Transformations **15**
5.1 Watershed . . . 15
5.2 AffineTransform . . . 15

6 Filters **16**
6.1 Gauss . . . 16
6.2 Kuwahara . . . 16
6.3 MedianFilter . . . 16

1 Loading Images

1.1 ImageReadTIFF

DESCRIPTION `ImageReadTIFF` function allows to read the image file (specifically, TIFF format). The value is stored in a variable and can be viewed with `Show()` function.

```
<variable_name> = ImageReadTIFF(<path of the file>)
<variable_name>.Show()
```

NOTE There are other functions for other file formats - such as `ImageReadICS` for ICS images, `ImageReadJPEG` for JPEG images, `ImageReadPNG` for PNG images, `ImageReadNPY` for numpy arrays, `ImageRead` incase the format is unknown.

RETURNS VALUE Returns the image

---

2 Histograms and Transformations

2.1 Histogram

DESCRIPTION This function allows to compute the histogram of a given image. The function takes the original image as the argument, and can be viewed with the help of `Show()` function. It also takes the number of bins (an integer) as an argument.

```
Histogram(<image>).Show()
Histogram(<image>, nBins=<int>).Show()
```

RETURNS VALUE Returns the histogram of the image

2.2 ContrastStretch

DESCRIPTION This function allows to contrast stretch the original image. The lower and upper bounds for contrast stretching are provided as the second and third argument for the function, with the original image being the first argument.

```
ContrastStretch(<image>, <lower bound>, <upper bound>)
```

NOTE The contrast-stretched image can be viewed using `Show()` function.

RETURNS VALUE Transformed image

2.3 HistogramEqualization

DESCRIPTION This function modifies the image such that its histogram is as flat as possible.

```
HistogramEqualization(<image>,nBins=256)
```

RETURNS VALUE None

2.4 HistogramMatching

NAME This function modifies the image such that its histogram is as close as possible, to a target histogram (a reference image can be used as an example).

```
HistogramMatching(<image>,Histogram=<example>)
```

NOTE The example histogram is usually a 1D histogram.

RETURNS VALUE dip Image

2.5 ColorSpaceManager

NAME Allows to manage the color space of the image

```
<variable_name>.ColorSpace() # allows to access the color space of the image
ColorSpaceManager.Convert(<variable_name>, "<new color space>")
```

DESCRIPTION ColorSpaceManager allows to manage and convert the color of an image from one space to another.

NOTE The colorspace of an image can be obtained by `ColorSpace()` function. This can be converted into another format by using `Convert()` function.
The available color spaces are RGB, sRGB, HSL, HSV, HSI, Lab, XYZ, Luv, YCbCr ,Ohta, YUV, LinearRGB.

RETURNS VALUE None

---

3 Segmentations

3.1 Threshold

DESCRIPTION This function thresholds the given image

```
<variable_name> = Threshold(<image>)
```

NOTE The thresholding done by default is Otsu. There are other thresholding functions such as `IsodataThreshold`,`TriangleThreshold`, `BackgroundThreshold`, `MinimumErrorThreshold`, `GaussianMixtureModelThreshold` , `VolumeThreshold` and `FixedThreshold`

RETURNS VALUE A tuple of two values. The first value provides the thresholded image, and the second value provides the threshold value.

3.2 IsodataThreshold

DESCRIPTION This function is used to perform Isodata Thresholding on the given image. The first argument for the function is the input image. The second argument is the mask. Only those pixels in mask are used to determine the histogram on which the Isodata algorithm is applied, but the threshold is applied to the whole image. The final argument is the `nThresholds`, which defines the thresholds. If it is set to 1, the output is a binary image.

```
<variable_name> = IsodataThreshold(<image>, <mask_image>, nThresholds = <int>)
```

NOTE The function works fine even with just the input image, and none of the other arguments.

RETURNS VALUE Transformed image

3.3 RangeThreshold

DESCRIPTION This function thresholds the image at two values, the lower and upper bound. The image is taken as the first argument while the lower and upper bounds are taken as the second and third arguments (which are float values).

```
<variable_name> = RangeThreshold(<image>, <lower bound value>, <upper bound value>)
```

RETURNS VALUE Transformed image

3.4 FixedThreshold

DESCRIPTION This function is similar to the threshold function. It takes the image as the first argument and a fixed threshold as the second argument.

```
<variable_name> = FixedThreshold(<Image>, <int>)
```

RETURNS VALUE Transformed image

3.5 VolumeThreshold

DESCRIPTION This function thresholds an image such that a fraction of the value is foreground. The function takes the image to be thresholded as the first argument, followed by the mask. Another argument `volumeFraction` determines the fraction of the pixels (a float value) that are the foreground.

```
<variable_name> = VolumeThreshold(<image>, <mask>, volumeFraction=<value>)
```

RETURNS VALUE Transformed image

3.6 HysteresisThreshold

DESCRIPTION This function allows to perform hysteresis threshold on the given image. The image to be thresholded is taken as the first argument, while the lower and upper thresholds are taken as the second and third arguments. It selects only the connected regions, which has one of the locations in the highThreshold regions.

```
<variable_name> = HysteresisThreshold(<image>, <lower threshold>, <upper threshold>)
```

RETURNS VALUE Transformed image

3.7 MorphologicalThreshold

DESCRIPTION This function is used to perform morphological thresholding on the given image. The first argument is the image to be thresholded. The structuring element is taken as the second argument (is a constant value). The edge type is given as the third argument (string value), which defines the type of edges and the funal argument is the `boundaryCondition`, which determines the boundary conditions (string value)

```
<variable_name> = MorphologicalThreshold(<input image>, se = <>, edgeType = <string>, boundaryCondition = <[list
of values]>)
```

NOTE The `edgeType` can be ***texture, object, both, dynamic***. The `boundaryCondition` can be a list of values. The most commonly used and meaningful one is ***add min***. Refer to the official documentation of Morphological Thresholding and Dilation (for boundaryCondition) for more clarity on how to provide appropriate input for the variables.

RETURNS VALUE dip Image

3.8 Entropy

DESCRIPTION This function calculates the entropy of a given image.

```
<variable_name> = Entropy(<image>, <mask>, nBins = <int> )
<variable_name> = Entropy(<image>)
```

NOTE There are two ways of using the function, as shown above. The first method allows to feed the image and mask as the first and second arguments, along with the number of bins to calculate the entropy from the histogram. The second method allows to calculate the entropy directly from the histogram (which is fed as the first argument). <nBins> is set by default to 256.

RETURNS VALUE Float value

3.9 OtsuThreshold

DESCRIPTION This function is used to threshold the image using Otsu's method (maximal inter-class variance method).

```
<variable_name> = OtsuThreshold(<image>, <mask>, nBins = <int>)
<variable_name> = OtsuThreshold(<image>)
```

NOTE The default value of `nBins` is set to 256.

RETURNS VALUE Transformed image

3.10 KMeansClustering

DESCRIPTION This function allows to perform K-Means clustering on a given image.
The function works on both the image, and the histogram. In case of passing a histogram, it is usually multi-dimensional in nature. The number of clusters is determined by `nClusters`.

```
<variable_name> = KMeansClustering(<image>, nClusters = <int>)
<variable_name> = KMeansClustering(<histogram>, nClusters = <int>)
```

NOTE This method can be used for multi-dimensional histograms. <nClusters> is set by default to 2. For 1D histograms, `IsodataThreshold` (Section 3.2) is more efficient, and deterministic.

RETURNS VALUE None

3.11 Label

DESCRIPTION This function is used to label the connected components in an image. The function can be utilized in two ways.

- It takes the image directly, as an argument and returns the labeled image.
- The second method is also shown below, where it takes the image as an argument. Additional factors such as `minSize` and `maxSize` are used, which is used to find the area of the smallest and largest object to be detected. `boundaryConditions` takes a list of values.

```
Label(<image>)
Label(<image>, connectivity = <int>, minSize = <int>, maxSize = <int>, boundaryConditions = <[list of values]>)
```

NOTE For more clarity on `boundaryConditions` and possible values, refer to the official DIPLib documentation on Boundary Conditions

RETURNS VALUE Labeled image

3.12 GradientMagnitude

DESCRIPTION This function computes the gradient magnitude of the given image. By default, the function uses Gaussian derivatives for computation. The value of `sigmas` determine the smoothing effect. The `boundaryCondition` function determines how the boundary should be expanded in each dimension. The `truncation` determines the approximation of the Gaussian kernel without unnecessary expense. The `process` variable allows to eliminate the dimension, along which no derivatives needs to be taken.

```
<variable_name> = GradientMagnitude(<image>, sigmas = <int>, method = '<string>', boundaryCondition = <[list of
images]>, process = <[list of images]>, truncation = <float>)
```

NOTE This is equivalent to a combination of functions `Norm(Gradient(<image>))`. If the value of sigma is 0 or negative, the dimensions are not processed. The default value of truncation is set to 3, which gives a good approximation. In the case of `process`, for a 3-dimensional image, if the image needs to be processed in the x-axis, then the value is set to `'true','false', 'false'`.

RETURNS VALUE Transformed image

3.13 SobelGradient

DESCRIPTION This function computes the Sobel derivative gradient filter of an image.

```
<variable_name> = SobelGradient(<image>, dimension=<int>, boundaryCondition= <list[str]>)
<variable_name> = SobelGradient(<image>, dimension=<int>)
```

NOTE `dimension` is set by default to 0.

RETURNS VALUE Transformed image

3.14 Laplace

DESCRIPTION This function computes the Laplacian of the image. The Laplacian of input image f is written as $\nabla \cdot \nabla f = \triangle^2 f = \nabla f$ and is given by

$$\nabla = \sum_i \frac{\partial^2}{\partial u_i^2} f$$

```
<variable_name> = Laplace(<image>, sigma=<float>, upper=<float>, method=<string>)
```

NOTE `sigma` is set by default to 0.1 and `method` is set by default to S::BEST.

RETURNS VALUE Transformed image

3.15 Canny

DESCRIPTION Detect edges in the grey-value image by finding salient ridges in the gradient magnitude

```
<variable_name> = Canny(<image>, sigmas=<float array>, lower=<float>, upper=<float>, selection=<string>)
```

NOTE By default, `lower` is set to 0.5, `upper` is set to 0.9, `sigmas` is set to {1} and `selection` is set to S::ALL.

RETURNS VALUE Transformed image

---

4 Mathematical Morphological Functions

4.1 Dilation

DESCRIPTION This function computes the dilation of an image and takes arguments: the image, the structuring element and a boundary condition.

```
<variable_name> = Dilation(<image>, <se>, boundaryCondition = <string array>)
<variable_name> = Dilation(<image>, <se>)
```

NOTE By default, `boundaryCondition` is set to {}.

RETURNS VALUE Transformed image

4.2 Erosion

DESCRIPTION This function computes the erosion of an image and takes arguments: the image, the structuring element and a boundary condition.

```
<variable_name> = Erosion(<image>, <se>, boundaryCondition = <string array>)
<variable_name> = Erosion(<image>, <se>)
```

NOTE By default, `boundaryCondition` is set to {}.

RETURNS VALUE Transformed image

4.3 Opening

DESCRIPTION This function computes the opening of an image and takes arguments: the image, the structuring element and a boundary condition.

```
<variable_name> = Opening(<image>, <se>, boundaryCondition=<string array>)
<variable_name> = Opening(<image>, <se>)
```

NOTE By default, `boundaryCondition` is set to {}.

RETURNS VALUE Transformed image

4.4 Closing

DESCRIPTION This function computes the closing of an image and takes arguments: the image, the structuring element and the boundary condition.

```
<variable_name> = Closing(<image>, <se>, boundaryCondition = <string array>)
<variable_name> = Closing(<image>, <se>)
```

RETURNS VALUE Transformed image

4.5 AreaOpening

DESCRIPTION This function computes the area opening of an image and takes arguments: image, masked image, the filter size, the connectivity and the polarity.

```
<variable_name> = AreaOpening(<image>, <mask>, filterSize=<int>, connectivity= <int>, polarity= <int>)
```

NOTE The connectivity is set by default to 0 and `filterSize` is set by default to 50 and `polarity` is set to 'opening'.

RETURNS VALUE Transformed image

4.6 AreaClosing

DESCRIPTION This function computes the area closing of an image and takes arguments: image, masked image, the filter size, connectivity and polarity.

```
<variable_name> = AreaOpening(<image>, <mask>, filterSize =<int>, connectivity= <int>, polarity= <int>)
```

NOTE `connectivity` is set by default to 0 and `polarity` is set by default to 'closing'.

RETURNS VALUE Transformed image

4.7 BinaryOpening

DESCRIPTION This function computes the binary opening of an image. It takes an DIP image and connectivity, and the number of iterations and the edge condition as arguments. The edge condition parameter specifies whether pixels past the border of the image should be treated as object (by passing "object") or as background (by passing "background").

```
<variable_name> = BinaryOpening(<image>, connectivity= <int>, iterations = <int>, edgeCondition=<string array>)
```

NOTE `connectivity` is set by default to -1, `iterations` is set to 3 and `edgeCondition` is set to S::SPECIAL.

RETURNS VALUE Transformed image

4.8 BinaryClosing

DESCRIPTION This function computes the binary closing of an image. It takes an DIP image and connectivity, the number of iterations and the edge condition as arguments.The edgeCondition parameter specifies whether pixels past the border of the image should be treated as object (by passing "object") or as background (by passing "background"). Additionally, you can set it to "special" for special handling: "background" for the dilation, "object" for the erosion; this avoids the border effect you can get in the corners of the image in some cases.

```
<variable_name> = BinaryClosing(<image>, connectivity= <int>, iterations = <int>, edgeCondition=<string array>)
```

NOTE `connectivity` is set by default to -1, `iterations` is set to 3 and `edgeCondition` is set to S::SPECIAL.

RETURNS VALUE Transformed image

4.9 GrowRegions

DESCRIPTION This function is used to grow (dilate) the labeled regions uniformly. It takes the image as the first argument, and the mask (optional) as the second argument. The variable `connectivity` determines the shape of the structuring element, and `iterations` determines the step count.

```
<variable_name> = GrowRegions(<image>, <mask>, connectivity=<int>, iterations = <int>)
```

NOTE `connectivity` is set by default to -1 and `iterations` is set to 0. If `iterations` is set to 0, the image is dilated until no further dilations are possible.

RETURNS VALUE Transformed image

4.10 SplitRegions

DESCRIPTION This function is used to ensure that there is a gap between regions with unequal labels.

```
<variable_name> = SplitRegions(<image>, connectivity= <int>)
```

NOTE The `connectivity` is set by default to 0.

RETURNS VALUE Transformed image

4.11 MorphologicalRange

DESCRIPTION This function computes the morphological range of an image. It implements a morphological edge detector based on the difference of two complementary morphological operations. These can be chosen through the edgeType parameter.

```
<variable_name> = MorphologicalRange(<image>, <se>, edgeType = <string array>, boundaryCondition= <string
array>)
```

NOTE `edgeType` is set by default as S::TEXTURE but can be one of:

- "texture": response is limited to edges in texture (i.e. scales smaller than the structuring element).
- "object": response is limited to object edges (i.e. scales larger than the structuring element).
- "both" or "dynamic": all edges produce equal response.

RETURNS VALUE Transformed image

4.12 Tophat

DESCRIPTION It computes the top-hat of an image. It takes two arguments, the image and the structuring element se.

```
<variable_name> = Tophat(<image>, <se>, edgeType = <string array>, polarity = <string>, boundaryCondition =
<string array>)
```

NOTE `edgeType` is sert by default to S::TEXTURE and `boundaryCondition` is set by default to {}. `polarity` can be either “white” to indicate objects are brighter than the background, or “black” to indicate objects are darker than the background. You can make a **blackhat filter** out of the tophat filter by setting the ‘polarity’ parameter to “black” in the ‘dip.Tophat()’ function, which causes it to compute the difference between the closing of the image and the original, highlighting dark features on a bright background.

RETURNS VALUE Transformed image

4.13 HitAndMiss

DESCRIPTION It computes the hit and miss of an image. It takes arguments, the image and the structuring element `se`, the hit structuring element, the miss structuring element, the mode and the boundaryCondition.

```
<variable_name> = HitAndMiss(<image>, <se>, hit = <structuring element>, miss = <structuring element>; mode =
<string>, ) <variable_name> = HitAndMiss(<image>, <se>)
```

NOTE `mode` is set by default to UNCONSTRAINED and the `boundaryCondition` is set by default to {}. The Hit-and-Miss transform, uses two structuring elements, hit must be within the structures, miss must be without.

For a binary image, the result is the intersection of the erosion of the image with hit and the erosion of the inverted image with miss.

For a grey-value image, there are two definitions of the operator. If mode is "unconstrained", the output is the difference of the erosion with hit and the dilation with miss, with any negative values clipped to 0.

If mode is "constrained", a more restrictive definition is applied (conditions evaluated pixel-wise):

- If in == erosion(in,hit) && dilation(in,miss) < in: out = in - dilation(in,miss).
- If in == dilation(in,miss) && erosion(in,hit) > in: out = erosion(in,hit) - in.
- Otherwise: out = 0.

RETURNS VALUE Transformed image

4.14 Metric

DESCRIPTION This function is used to establish the connectivity for distance transforms and neighbor-based operations. The function can be used in two ways - as shown below. The first method takes just the input image. The second method takes additional arguments such as `PixelSize`.

```
Metric(<string>,<image>)
Metric(<string>, PixelSize = <>
```

NOTE There are multiple metrics such as `Chamfer`, `city` and `chess`. Refer to the official DIPLib documentation for more values for the argument for Metric strings

RETURNS VALUE None

4.15 VectorDistanceTransform

DESCRIPTION This function computes the Euclidean vector distance transform of an image and returns an output in the form of the vector components of an image.

```
<variable_name> = VectorDistanceTransform(<image>, border = <string>, method = <string>)
```

NOTE `border` is set by default to S::BACKGROUND and `method` S::FAST.

RETURNS VALUE Vector components of image

4.16 ConvexHullInterval2D

DESCRIPTION This function computes the 2D Convex Hull.

```
<variable_name> = ConvexHullInterval2D()
```

NOTE Use with dip.Thickening to grow concave sections of objects until the objects are all convex. This leads to octagonal shapes.

RETURNS VALUE A list

4.17 Thickening

DESCRIPTION This function applies the thickening operator on an image. The operation is applied with each of the intervals in intervals, and repeated iterations times. If iterations is 0, the operation is repeated until convergence.

```
<variable_name> = Thickening(<image>, <intervals>, iterations= <int>)
```

RETURNS VALUE Thickened image

4.18 MeasurementTool

DESCRIPTION MeasurementTool is a function which allows to compute various measurements of the objects in the image. The official DIPLib website has a list of Measurements and the descriptions that can be measured using the function. . This is used in conjunction with Measure function (Section 4.19) to obtain the values.

4.19 Measure

DESCRIPTION This function computes all measurements of objects.

```
<variable_name> = Measure(<labeled_image>,<grey_image>, [features])
```

NOTE It takes a labeled image and the original (grey image) and a set of features such as 'Center', 'Size' and 'Mean' to measure.

RETURNS VALUE List of measurements as an array

4.20 EdgeObjectsRemove

DESCRIPTION This function removes edge objects from a binary or labeled image.
If the input image is labeled, this function erases the labels that touch the edge. In this case, connectivity is ignored.

```
<variable_name> = EdgeObjectsRemove(<image>, connectivity=<int>)
```

NOTE By default `connectivity` is set to 1.

RETURNS VALUE Filtered image

---

5 Transformations

5.1 Watershed

DESCRIPTION This function allows to perform Watershed segmentation on the chosen image. There are two ways of using the function, as shown below. The mask (binary image) can also be used, but is optional (the function works with or without the mask). The third argument is `connectivity` which determines which pixels are considered neighbors. The fourth argument is `maxDepth` which controls the depth of the watershed basins (an int value).

```
<variable_name> = Watershed(<image>, <mask>, connectivity = <int>, maxDepth=<int>, maxSize = <int>, flags =
<string set>)
<variable_name> = Watershed(<image>, maxDepth=<int>)
```

NOTE The default value of `connectivity` is set to 1, `maxDepth` is set to 1 and `maxSize` is set to 0.

RETURNS VALUE Transformed image

5.2 AffineTransform

DESCRIPTION This function applies an arbitrary transform on a 2D or 3D image. The transform needs to be inputed as a matrix in the second argument. A linear transformation matrix containing 4 values is in 2D and a linear transformation matrix containing 9 values is in 3D.

```
<variable_name> = AffineTransform(<image>, <transformation matrix>, interpolationMethod = <string array>)
```

NOTE `interpolationMethod` is set by default to S::LINEAR.

RETURNS VALUE Transformed image

---

6 Filters

6.1 Gauss

DESCRIPTION This function applies a Gaussian Filter on the original image. The kernel size is changed by the second argument (which takes the size of the kernel as an integer). The filtered image can then, be viewed using the `Show()` function.

```
<variable_name> = Gauss(<image>, kernel = <kernel>)
```

RETURNS VALUE Filtered image

6.2 Kuwahara

DESCRIPTION This function applies a Kuwahara Filter on the original image. The kernel size is changed by the second argument (which takes the size of the kernel as an integer). The filtered image can then, be viewed using the `Show()` function.

```
<variable_name> = Kuwahara(<image>, kernel = <kernel>, theshold = <float>, boundaryCondition = <string array>)
```

NOTE `threshold` is set by default by 0.0.

RETURNS VALUE Filtered image

6.3 MedianFilter

DESCRIPTION This function applies a Median Filter on the original image. The kernel size can be varied by the second argument which takes the size of the kernel as as an integer. The filtered image can then, be viewed using the `Show()` function.

```
<variable_name> = MedianFilter(<image>, kernel = <kernel>, boundaryCondition = <string array>)
```

RETURNS VALUE Filtered image

Index of Functions

A

AffineTransform 15
AreaClosing 10
AreaOpening 10

B

BinaryClosing 11
BinaryOpening 11

C

Canny 9
Chamfer 13
Closing 10
ColorSpaceManager 4
ContrastStretch 3
ConvexHull 14

D

Dilation 9

E

EdgeObjectsRemove 15
Entropy 6
Erosion 9

F

FixedThreshold 5

G

Gauss 16
GradientMagnitude 8
GrowRegions 11

H

Histogram 3
HistogramEqualization 4
HistogramMatching 4
HitAndMiss 13
HysteresisThreshold 6

I

ImageReadTIFF 3
IsodataThreshold 5

K

KMeansClustering 7
Kuwahara 16

L

Label 7
Laplace 8

M

Measure 14
MeasurementTool 14
MedianFilter 16
MorphologicalRange 12

MorphologicalThreshold 6

O

Opening 10
OtsuThreshold 7

R

RangeThreshold 5

S

SobelGradient 8
SplitRegions 12

T

Thickening 14
Threshold 4
Tophat 12

V

VectorDistanceTransform 13
VolumeThreshold 6

W

Watershed 15